ICS 2026 BombLab 实验报告

一、实验目标与总体方法

本实验通过分析个人 bomb 的 x86-64 汇编，恢复每个 Phase 对应的参数化输入关系，并实现依据 challenge 动态生成答案的求解器。

流程为：使用 objdump 反汇编并定位函数；结合寄存器传参和内存偏移追踪数据流；将算术、移位、异或、循环、分支和递归还原为 C 级关系；在 defuser.c 中实现；运行 bomb 与公开测试验证；分析 Secret Phase 的前置门控与上下文混合。

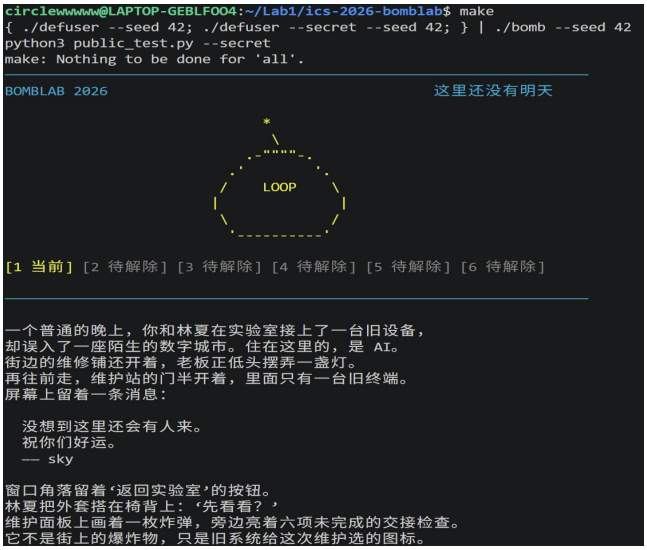
二、challenge 与整数语义

求解器接收 64 位 challenge。中间变量多为 uint32_t，因此赋值时按模 2^32 截断；无符号右移为逻辑右移；rol32/ror32 分别表示 32 位循环左移与循环右移。seed 到 challenge 的生成采用 SplitMix64 风格混合过程：

```
x = seed + 0x9e3779b97f4a7c15 * (phase + 1)
x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9
x = (x ^ (x >> 27)) * 0x94d049bb133111eb
challenge = x ^ (x >> 31)
```

因此每关答案依赖当前 challenge，不能将某次运行的固定答案硬编码。

三、关卡思路、运行与测试

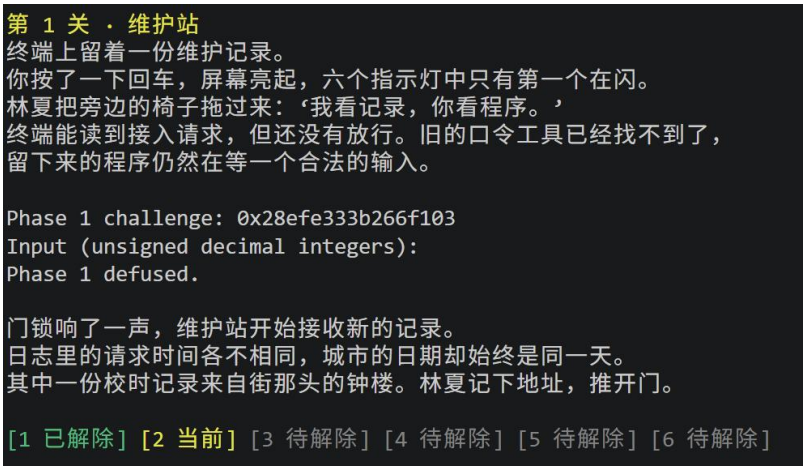


Phase 1：三段混合计算

将 challenge 拆为高 32 位 hi、中间移位字段 mid 和低 32 位 lo。三个步骤分别执行线性组合、异或和加法，得到一个 32 位结果。

```
hi  = (uint32_t)(c >> 32)
mid = (uint32_t)(c >> 16)
lo  = (uint32_t)c
s1 = (hi + 2*mid) ^ 0xc1da
s0 = (3*lo) ^ (hi + 0x7fa8)
ans = 2*s0 + (s1 ^ 0x8b05)
```

推理：移位与截断对应 hi、mid、lo；imul/lea 对应乘法或线性组合；xor 对应按位异或。输出一个无符号十进制整数。正式版应从 phase_01.S 摘录相应原始指令，以形成汇编与公式的直接对应。



Phase 2：六项递推序列

Phase 2 生成六个整数。首项来自 challenge 第 18 位起始字段，乘 2 后取 8 位并加 29；后续各项从不同 4-bit nibble 取值，与序号异或，再加前一项的 4 倍。

```
v[0] = (((challenge >> 18) * 2) & 0xff) + 29
for i = 1..5:
    shift = 5 + 3*(i-1)
    nibble = (challenge >> shift) & 0xf
    v[i] = (nibble ^ i) + 4*v[i-1]
```

推理：循环变量 i 决定每轮字段偏移；前一项乘 4 可由移位或 lea 实现；异或和加法形成递推。输出六个空格分隔的无符号十进制整数。

```
第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x47526757130f9f52
Input (unsigned decimal integers):
Phase 2 defused.

校时检查通过了。钟楼送出一条新记录，几秒后又收回原样的一条。
‘不是钟没走。’林夏把两条记录放在一起，‘它送出去又回来了。’
你们沿着线路往下查，重复的地址最后指向街口的分流站。

[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]
```

Phase 3：索引选择的分支表

Phase 3 提取 which = challenge 低 3 位（0–7），并计算 x = (challenge >> 32) XOR (challenge >> 3)。根据 which 进入 8 个分支之一，输出 which 与计算值两个整数。

```
which = challenge & 7
x = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3)
case 0: y = 5*x + 0x1aea
case 1: y = 5*x + 0xfa45
case 2: y = (x ^ 0xd104) + (x >> 1)
case 3: y = (2*x) ^ 0x9b64
case 4: y = 5*x + 0x17f43
case 5: y = 3*x + 0x2bd2
case 6: y = (2*x) ^ 0x5f36
default: y = (x ^ 0x5cb3) + (x >> 2)
output = "which y"
```

推理：比较跳转或间接跳转形成 switch 分支；逐条记录每条路径的常数、乘数和移位量，不能将不同 case 合并为同一表达式。

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x581ce1ff0e4ae394
Input (unsigned decimal integers):
Phase 3 defused.

这次请求没有再绕回来。分流站把它送进档案馆，收到一条回复：
‘未找到交接存档。’回复后面附着一份旧目录。
林夏收好路线图。你们穿过马路，档案馆的门刚刚打开。

[1 已解除] [2 已解除] [3 已解除] [4 当前] [5 待解除] [6 待解除]
```

Phase 4：二分递归路径编码

Phase 4 取 target = challenge 低 7 位，key = (challenge >> 7) XOR (challenge >> 32)。递归区间初始为 [0,127]，每层计算 mid。命中时返回 (mid+0xf5) XOR key；向右回溯异或 0x0c，向左回溯异或 0x0f，每层还执行乘 5 和 mid+key 混合。

```
mid = low + ((high-low) >> 1)
if mid == target: return (mid + 0xf5) ^ key
if target > mid:
    sub = walk(target, mid+1, high, key)
    return ((sub*5) ^ (mid+key)) ^ 0x0c
else:
    sub = walk(target, low, mid-1, key)
    return ((sub*5) ^ (mid+key)) ^ 0x0f
```

推理：必须恢复递归返回后的路径编码；只计算命中的 mid 而忽略回溯阶段会得到错误结果。

```
第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

Phase 4 challenge: 0x09bc585a244823f2
Input (unsigned decimal integers):
Phase 4 defused.

最后一层索引返回了一份完整存档，还有那次迁移留下的记录。
新设备接上时，交接没有做完。城市只能反复恢复最后一份旧存档；
居民今天做过的事，到了夜里就都没了。
存档后面列着几处还没核对的设备。你认出了维修铺那盏灯。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 当前] [6 待解除]
```

Phase 5：链表遍历与节点字段混合

Phase 5 提取 index = challenge & 7、steps = (challenge >> 3) & 3，以及 x = (challenge >> 6) XOR (challenge >> 32)。初始值由起始节点 value、tag 和常数 0x4de3 混合；沿 next 遍历 steps 次，每次执行 value = (value XOR tag) + node.value。输出累计 value 和最终节点 value。

```
index = challenge & 7
steps = (challenge >> 3) & 3
x = (challenge >> 6) ^ (challenge >> 32)
value = node[index].value
value = (x ^ node[index].tag ^ 0x4de3) + value
repeat steps:
    current = node[current].next
    value = (value ^ node[current].tag) + node[current].value
output = "value node[current].value"
```

节点只读数据转录表：

idx	next	value	tag
0	1	633	5036
1	2	2920	61812
2	3	2128	575
3	4	877	48231
4	5	1621	9375
5	6	1129	44901
6	7	1302	23128
7	8	2597	55717
8	9	1834	31008
9	10	2389	3199
10	11	109	54226
11	NULL	293	54554

关键调试：最初节点结构体字段顺序与宽度判断错误，造成 next、value、tag 读取错位。结合指针访问指令、地址及内存数据，确认 next 为指针字段，value 和 tag 为两个 32 位字段；修正结构体和节点映射后通关。

```
第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0xde4431fa3c80db06
Input (unsigned decimal integers):
Phase 5 defused.

最后一处连接也对上了，今天的设备状态第一次进入待保存的记录。
维修铺老板试着拧了一下灯座，灯亮了一瞬，又被他小心地关掉。
‘留到明天装好。’他说。
交接清单上只剩最后一项。你们带着记录走到回廊尽头。

[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 当前]
```

Phase 6：两级操作函数组合

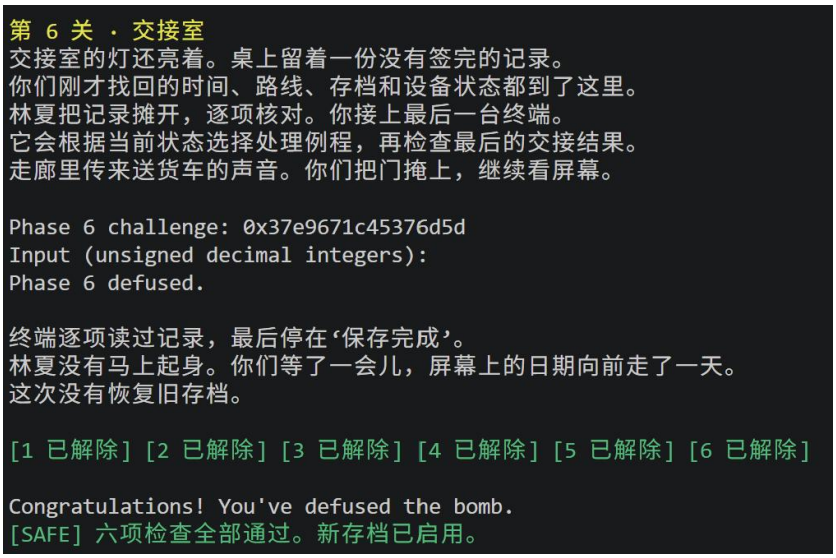
Phase 6 从 challenge 提取两个操作选择器 which1、which2 和输入 x1。先计算 p6_op(which1,x1)，异或 0xf35a，再送入第二个操作函数，最后加 0x8d74。which 值 5、6、7 均进入 default 分支。

```
which1 = challenge & 7
x1 = (challenge >> 6) ^ (challenge >> 32)
first = p6_op(which1, x1) ^ 0xf35a
which2 = (challenge >> 3) & 7
answer = p6_op(which2, first) + 0x8d74

case 0: (x ^ (x >> 2)) + 0xeef0
case 1: ror32(x, 3)
case 2: ((x << 5) + 0x0dd0) ^ (3*x)
case 3: (x ^ 0x6959) + (x << 7)
```

```
case 4: (x + 0xac7f) ^ (x >> 7)
default: rol32(x, 1) ^ 0x4c11
```

推理：第二次运算的输入是第一次运算并异或 0xf35a 后的 first，而非原始 x1；循环移位与普通移位必须区分。



Secret Phase：门控、上下文与三字段校验

Secret Phase 依赖前六关答案构成的 transcript context。主程序先计算六关答案上下文，再用 phase 7 challenge 调用 secret_gate。secret_gate 检查长度和固定 OPEN 前缀，再进入 secret_phase 验证三个无符号整数。

secret_mix 位于 0x402620：rol eax,7；add eax,0x682e；xor eax,0xf0bb。secret_phase 调用 valid_u32_fields，要求三个字段成功解析，并检查第一个字段等于 context 低两位。

```
c  = (uint32_t)context & 3
t1 = high32(challenge) ^ low32(challenge)
t2 = high32(context) ^ low32(context)
r9 = (c * 0x682e) ^ (context >> 16)
b  = secret_mix(t1 ^ t2) ^ r9
a  = c
d  = b ^ (t2 + c + 0xf0bb + (t1 >> 7))
input = 32*'A' + "OPEN" + decimal(a,b,d)
```

secret_gate 位于 0x402720 附近，调用 strlen、secret_copy，并比较 0x4e45504f（小端字节为 OPEN）；secret_phase 位于 0x402640，调用 valid_u32_fields 并解析三个字段。transcript_context 从 0x243f6a8885a308d3 开始，按 Phase 1–6 顺序混合 phase 编号及答案中的十进制字段。因此 Secret Phase 不能脱离前六关独立硬编码。求解器使用 32 个 A 作为前缀，后接 OPEN 和三个十进制字段。

```
● Honor · 观测台旧备份

  林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
  文件夹叫‘留给后来的人’，署名是 sky 和许遥。
  扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
  这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
  桌上的阅读器仍在等待旧程序认可的输入。
  城市已经安全了。是否继续读下去，由你们自己决定。

  Secret phase defused.
  [SAFE] 城市照常运行。附加档案已恢复。
```

```
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

每个已完成关卡使用公开列表以外的 seed 验证的记录

```
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1/ics-2026-bomblab$ mkdir -p evidence/logs

for p in 1 2 3 4 5 6; do
    echo "===== Seed 3 / Phase $p ====="
    ./defuser --seed 3 --phase "$p" \
        2>&1 | tee "evidence/logs/seed3_phase${p}.log"
done
===== Seed 3 / Phase 1 =====
366957477
===== Seed 3 / Phase 2 =====
69 285 1155 4628 18519 74089
===== Seed 3 / Phase 3 =====
7 2017169167
===== Seed 3 / Phase 4 =====
3560310880
===== Seed 3 / Phase 5 =====
2130421874 2597
===== Seed 3 / Phase 6 =====
2547422669
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1/ics-2^C6-bomblab$
circlewwwww@LAPTOP-GEBLFOO4:~/Lab1/ics-2026-bomblab$ ./defuser --secret --seed 3 \
    2>&1 | tee evidence/logs/seed3_secret.log
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN2 3429229678 937011916
```

四、调试过程中的问题与解决

Phase 5 的结构体布局错误是主要调试问题。最初对节点字段顺序和宽度判断不准确，导致链表链接与两个 32 位字段读取错位。通过查看指针访问指令、节点地址及内存数据，修正结构体定义和节点映射后，Phase 5 通过。

Secret Phase 的难点在于识别多层验证：OPEN 前缀门控、三个数字字段解析、与前六关答案绑定的 transcript context，以及 secret_mix 的循环移位、加法和异或关系。恢复这些关系后，按 32 位无符号语义实现求解器并通关。